VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
UNIVERSITY OF INFORMATION TECHNOLOGY
FACULTY OF SOFTWARE ENGINEERING



OBJECT-ORIENTED PROGRAMMING

LAB EXERCISE 3

Ho Chi Minh City, 09/2026

PRACTICAL CONTENT

SUBMISSION REQUIREMENTS AND INSTRUCTIONS

Submission instructions:

- Submit a .PDF file named STUDENTID_LAB3.PDF (for example, 18520412_LAB3.PDF).
- The .PDF report must follow the template provided on Courses.
- Include screenshots of the source code. For each method, specify its input, output, and algorithmic approach.
- Upload all Java source files to Drive and insert the sharing link in the report template.

Note: Submissions that do not satisfy the above requirements will not be graded.

LAB 3: ENCAPSULATION AND INHERITANCE

This lab introduces two fundamental principles of object-oriented programming in Java: encapsulation and inheritance. Students will practice access modifiers, getters and setters, static and instance members, and three inheritance structures before applying them to management problems.

1. Encapsulation

Encapsulation bundles the state and behavior of an object in a class and limits direct access to its internal data. Access to private attributes is typically provided through public operations that can enforce validation rules.

- Hide implementation details and prevent uncontrolled changes to object state.
- Validate changes through methods instead of exposing fields directly.
- Improve maintainability by keeping data-management rules inside their class.

Example: A bank account stores its balance privately and provides controlled deposit and withdrawal methods.

```
class BankAccount {
    private double balance;

    public BankAccount(double initialBalance) {
        balance = Math.max(0, initialBalance);
    }

    public double getBalance() { return balance; }

    public void deposit(double amount) {
        if (amount > 0) balance += amount;
    }

    public void withdraw(double amount) {
        if (amount > 0 && amount <= balance) balance -= amount;
    }
}
BankAccount account = new BankAccount(1000);
account.deposit(500);
account.withdraw(300);
System.out.println(account.getBalance()); // 1200.0
```

The balance cannot be modified using account.balance because the field is private. This example additionally validates the initial balance; it is a small improvement over the illustrative constructor in the source Lab 3.

2. Access Modifiers

Access modifiers define where a class member can be accessed. When no access modifier is declared, Java uses package-private (default) access.

Modifier	Same class	Same package	Subclass in another package	Other class in another package
private	Yes	No	No	No
default	Yes	Yes	No	No
protected	Yes	Yes	Yes*	No
public	Yes	Yes	Yes	Yes

* Protected access from a subclass in another package has Java-specific restrictions and is not equivalent to unrestricted access through any superclass object.

```
class Animal {
    public String name = "Lion";
    private int age = 5;
    protected String type = "Mammal";
    String habitat = "Savanna"; // package-private
}
```

3. Getter and Setter Methods

A getter returns the value of an attribute. A setter updates it, potentially after validating the input. Not every attribute needs a setter, especially when it should be read-only.

```
class Student {
    private String name;
    private int age;

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public int getAge() { return age; }

    public void setAge(int age) {
        if (age > 0) this.age = age;
        else System.out.println("Age must be positive!");
    }
}
Student s = new Student();
s.setName("Alice");
s.setAge(20);
System.out.println(s.getName() + " - " + s.getAge());
// Alice - 20
```

4. Static and Non-Static Members

A static member belongs to the class and is shared among its objects. An instance (non-static) member belongs to a particular object.

```
class MathOperations {
    static int count = 0;
    int instanceVariable = 10;

    public static int add(int a, int b) { return a + b; }
    public void showValue() { System.out.println(instanceVariable); }
}
System.out.println(MathOperations.add(5, 10)); // 15
MathOperations.count++;
MathOperations a = new MathOperations();
MathOperations b = new MathOperations();
a.instanceVariable = 20;
a.showValue(); // 20
b.showValue(); // 10
```

5. Inheritance

Inheritance allows a subclass to reuse accessible fields and methods of a superclass. The Java keyword extends establishes an IS-A relationship. A subclass can also introduce additional members or override inherited methods.

```
class Animal {
    void eat() { System.out.println("This animal eats food."); }
}
class Dog extends Animal {
    void bark() { System.out.println("The dog barks."); }
}
// Usage:
Dog dog = new Dog();
dog.eat();
dog.bark();
```

6. Types of Inheritance

6.1. Single Inheritance: One child class extends one parent class, such as Dog extends Animal.

6.2. Multilevel Inheritance: A class inherits from another subclass, forming a chain such as Grandparent -> Parent -> Child.

```
class Grandparent {
    void greet() { System.out.println("Hello from Grandparent!"); }
}
class Parent extends Grandparent {
    void sayHello() { System.out.println("Hello from Parent!"); }
}
class Child extends Parent {
    void introduce() { System.out.println("Hello from Child!"); }
}
```

6.3. Hierarchical Inheritance: Several subclasses extend one parent class, such as Car and Bike both extending Vehicle.

```
class Vehicle {
    void start() { System.out.println("Vehicle is starting..."); }
}
class Car extends Vehicle {
    void drive() { System.out.println("Car is driving..."); }
}
class Bike extends Vehicle {
    void ride() { System.out.println("Bike is riding..."); }
}
```

Note: A Java class cannot extend multiple classes. Java supports implementing multiple interfaces through implements.

7. EXERCISES

Exercise 1: University Student Graduation Management

Problem: A university manages students from two programs: College and University. Design classes that reuse the information common to both programs and provide program-specific graduation checks.

Common student information: Student ID, full name, address, total credits, and GPA.

- College students also have a graduation exam score.
- University students also have a thesis title and thesis score.

Program	Minimum credits	Minimum GPA	Additional condition
College	120	5.0	Graduation exam score >= 5.0
University	170	5.0	Thesis score >= 5.0

Requirements:

- Input a list containing students from both programs.
- Count the students who satisfy their program graduation criteria.
- Find the University student with the highest GPA.
- Use a common superclass with encapsulated fields and subclass-specific methods.

Suggested test: A College student with 125 credits, GPA 6.5 and exam score 7.0 is eligible. A University student with 165 credits, GPA 8.0 and thesis score 9.0 is not eligible.

Exercise 2: Ticket Management for Dam Sen Park

Problem: The park sells a Full Package Ticket and a Partial Ticket. Each ticket stores a ticket ID, holder name, year of birth, and number of games played.

Ticket type	Pricing rule	Conditions
Full Package	200,000 VND	Access to all 30 games
Partial	70,000 + 20,000 x games	70,000 VND entrance fee

Requirements:

- Input a list of tickets of both types.
- Calculate total revenue from all tickets sold.
- Count the number of Partial Tickets sold.
- Model shared ticket information in a superclass and pricing in subclasses.

Example: One Full Package Ticket and one Partial Ticket with 4 games generate 200,000 + (70,000 + 4 x 20,000) = 350,000 VND.

Exercise 3: Hotel Room Rental Management

Problem: Design classes for hotel guests, staff, and rental rooms. Create a Person superclass with common information and derived Guest and Staff classes.

Guest information: Citizen ID card (CCCD), full name, age. Staff information: License ID, employee ID, full name, age.

Room type	Rental price per day
VIP1	900,000 VND
VIP2	700,000 VND
VIP3	500,000 VND

Requirements:

- Create the Person, Guest, Staff, and Room classes.
- Implement operations to add and remove guests.
- Input the room type and number of rental days.
- Calculate rental cost as daily room price multiplied by the number of days.

Example: A VIP2 room rented for 3 days costs 700,000 x 3 = 2,100,000 VND.

Exercise 4: Singer Salary Management for XYZ Company

Problem: Regular and Famous singers share common information: full name, years of experience, number of albums sold, and number of performances. Famous singers also record the number of game shows attended.

Salary calculation (all amounts in VND):

Regular = 3,000,000 + 500,000 x years + 1,000 x albums + 200,000 x performances.

Famous = 5,000,000 + 500,000 x years + 1,200 x albums + 500,000 x performances + 500,000 x game shows.

Requirements:

- Input singers into a single array or list of the common superclass type.
- Calculate salary according to each singer category.
- Find the highest-paid singer; any one may be returned in case of a tie.
- Use inheritance and clearly define and implement every class.

Suggested design: Singer is the superclass; RegularSinger and FamousSinger override calculateSalary().

Exercise 5: Cosmetic Store Order Management

Problem: A store manages customer orders that contain predefined product packages. Each order stores order ID, customer information, invoice date, order price, and package information.

Customer information: customer ID, full name, address, and phone number.

- Romantic Package: Must include Rose Perfume; Shampoo and Shower Gel are optional.
- Fresh-Air Package: Must include Oily Hair Shampoo; Perfume and Shower Gel are optional.

Product / variation	Unit price (VND/ml)
Rose Perfume	20,000
Chamomile Perfume	15,000
Dry Hair Shampoo	20,000
Oily Hair Shampoo - Standard 1	30,000
Oily Hair Shampoo - Standard 2	40,000

Dry Skin Shower Gel	20,000
Oily Skin Shower Gel	40,000

Every product stores an ID, a type, a volume (ml), and a price. Calculate product prices from volume and the corresponding rate; calculate the package price from the included products.

Requirements:

- Input N orders with their customers and selected packages.
- Display all orders before saving.
- Save the complete order list to order_list.txt.
- Use encapsulation and inheritance to represent shared and specialized product/package behavior.

Example: A Romantic Package with 50 ml of Rose Perfume and 100 ml of Dry Skin Shower Gel costs 50 x 20,000 + 100 x 20,000 = 3,000,000 VND.

8. IMPLEMENTATION NOTES

The following recommendations are provided to clarify how students should implement the original Lab 3 requirements:

- Use private fields and expose only the operations needed to access or update values.
- Use meaningful constructors, getters, and validation rules for amounts, ages, scores, and quantities.
- When subclasses use a common operation with different rules, consider method overriding and polymorphism.
- Use Scanner to read console input and an array or ArrayList to store mixed subclass objects.
- For Exercise 5, use standard Java file-writing APIs and check for possible I/O errors.
- Prepare small test cases for normal inputs and boundary conditions.